DRIFT-SENSE / PHASE 2

R3 Camera Branch

Work, results and validation handoff

Prepared for the validation member (R5) | 12 November 2026

R3 implementation and its planned experiments are complete. The camera approaches tested here do not outperform the existing Lasso baseline. Combined IMU-camera evaluation is still pending.

R3 adds camera data to the existing two-flight study. It creates small, time-matched image inputs, tests ten camera summary measurements with the Phase 1 model (Tier 1), and trains a small camera-only image model (Tier 2 / E5).

Approach	Mean error MAE (cm)	R^2	Evidence
Phase 1 Lasso, reproduced	2.50	0.756	Two held-out flights
Lasso + ten camera inputs	3.63	0.325	Two held-out flights
Camera-only E5	4.42 ± 0.12	0.074 ± 0.045	Two flights, three seeds
Phase 1 mean baseline	4.53	-0.032	Stored Phase 1 reference

Smaller MAE is better. These are errors in predicting drift, not the drone's actual drift. The overall score gives both test flights equal weight. E5 ± values are sample standard deviations over three per-seed, equal-flight averages; they are not confidence intervals. [S1-S3]

What is ready

• Camera inputs for all 1,312 existing two-second windows; all 6,722 images verified.

• Ten prediction CSVs: four Tier 1 files and six E5 files, with the shared four-column format.

• All 16 R3 tests passed; the complete repository suite passed 85 tests in the implementation session. The notebook is fully executed.

• Source files, processed data, training curves, report material and instructions are available in the working tree.

Snapshot: repository base 8488cd6, including uncommitted R3 work. This report records the implementation results; independent validation by R5 remains the next step.

1. What R3 implements

Existing sensor windows are reused without regenerating labels or changing their order. Camera timestamps are matched using exact integer nanoseconds. Each window includes its start time and excludes its end time: [t_start_ns, t_end_ns).

Flight	Images opened	Windows	Images/window
MH_01_easy	3682	720	40
MH_02_easy	3040	592	40

Both streams run at approximately 20 Hz. Original 752 x 480 images are converted once to 64 x 64 grayscale. Missing images, invalid timing or large gaps produce a failure rather than borrowed frames or dropped windows. [S4]

Tier 1: describe image changes using ten numbers

Use all 40 images in each window, comparing its 39 adjacent pairs. Seven values describe the amount, direction and variation of apparent image movement. Two describe sharpness and one counts distinct corners. Only pairs wholly inside the window contribute. These values are appended to the original 104 sensor measurements, producing 114 inputs.

Tier 2: learn from beginning, middle and end images

Select the first frame, the frame nearest the midpoint and the last frame inside the window. Three image-processing blocks learn a compact 64-number description. A final layer predicts drift magnitude. The model has 23,873 adjustable parameters, including its 23,808-parameter image encoder.

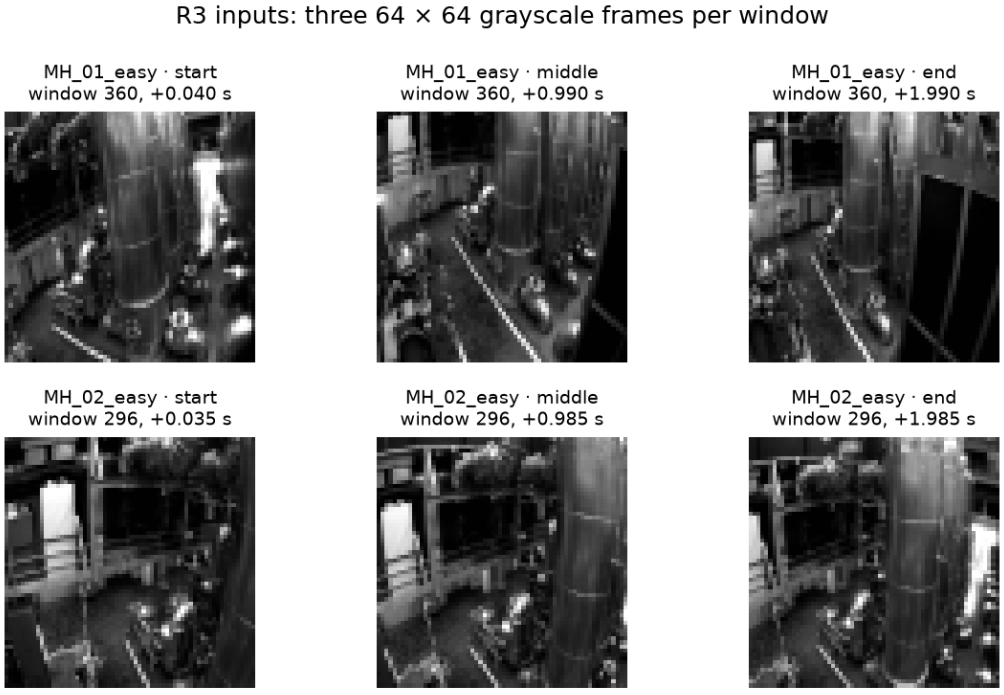


Figure 1. Actual 64 x 64 inputs. The three channels represent time, not red/green/blue. Examples are the middle window of each flight; they were not selected using labels.

Prepared window arrays are uint8 with shapes (720, 3, 64, 64) and (592, 3, 64, 64). R1's existing dataset converts them to float32 in [0, 1]. They total about 16.1 MB.

2. Evaluation setup and safeguards

The prediction target is err_mag_m: the magnitude of the existing two-second endpoint error. Each test flight is excluded from learning. The two flights swap roles between runs. Nearby windows overlap because the stride is 0.25 seconds; they are not 1,312 independent flight recordings.

Test flight	Training flight	Tier 1 train	E5 train	E5 check	Test rows
MH_01	MH_02	592	450	119	720
MH_02	MH_01	720	553	144	592

The two training protocols

Tier 1 calls the unchanged Phase 1 Lasso fitting code. It tests settings on five time blocks inside the training flight, removes 16 nearby rows around each held-out block, and selects the setting with the best median error. Scaling is fitted within each training block. It then refits on the whole training flight.

E5 uses R1's fixed split: the last 20% of the training flight checks progress, with at least four seconds between training-window endpoints and checking-window starts. This excludes 23 nearby windows per training flight. Checking data selects the saved model; the test flight is evaluated afterward. [S5]

> **Comparability limit: the test flights and target are the same, but E5 learns from fewer windows because it retains an inner checking block. Report the training counts alongside comparisons.**

E5 setting	Recorded value
Input / output	Three 64 x 64 grayscale frames / one drift magnitude
Image model	Three 3 x 3 stride-2 blocks, 16/32/64 channels; GroupNorm, ReLU, Dropout2d; global average pooling
Loss / optimiser	Mean squared error / AdamW
Learning rate / weight decay	0.001 / 0.0001
Dropout / batch size	0.3 / 32
Training cap / stopping	100 rounds; patience 15; minimum improvement 0.00001
Learning-rate schedule	Halve after five non-improving validation rounds
Repeated runs / device	Seeds 42, 43, 44 / CPU

Camera feature extraction reads no ground-truth labels, physics values or IMU readings. Labels are used only as answers for fitting/scoring. E5 explicitly enables camera input; the top-level use_camera=false remains for R2's IMU-only runner. No camera augmentation is used. [S6]

3. Tier 1 results and interpretation

This experiment asks whether ten image measurements improve the existing sensor-feature model. Each variant chooses its own penalty strength (alpha) using the training flight only. Test data does not choose that value.

Test	Inputs	Alpha	MAE cm	RMSE cm	R^2	Bias cm
MH_01	104	0.002154	2.59	3.28	0.676	-0.04
MH_01	114	0.001000	2.62	3.30	0.673	-0.09
MH_02	104	0.000464	2.42	3.18	0.837	-0.20
MH_02	114	0.100000	4.65	7.94	-0.022	+1.17

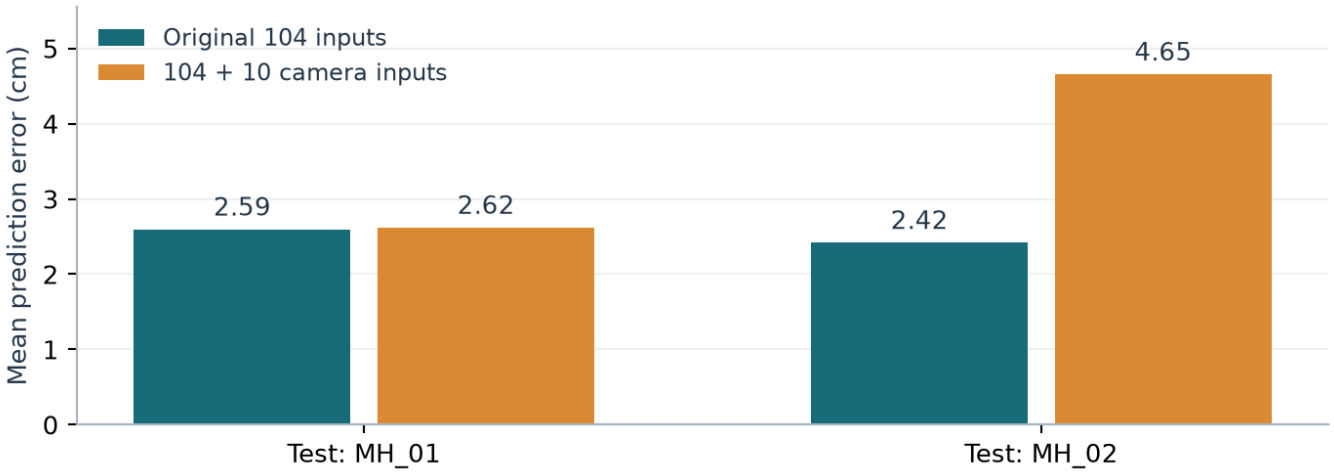


Figure 2. Adding the ten camera measurements increases prediction error on both test flights, with a much larger change on MH_02. Source: the four Tier 1 prediction files.

Observed findings

• The 104-input baseline reproduces the stored Phase 1 Lasso scores: mean MAE 2.50 cm and mean R^2 0.756.

• With camera measurements, mean MAE is 3.63 cm: an increase of 1.13 cm across the two folds.

• On the MH_02 test fold, training-only selection chooses alpha=0.1. Its saved predictions are constant and match the training-flight mean baseline. The large error increase is therefore associated with a heavily simplified model.

• On MH_01, the change is small: 2.59 to 2.62 cm. No seed spread is supplied for deterministic Lasso.

Interpretation: these ten measurements, together with the existing fitting rule, do not improve this two-flight baseline. This does not establish that camera information is useless in every representation or in a combined model. [S1-S3]

4. Camera-only E5 results

All six runs are reported. Best round means the weights retained using the checking data; it is not selected using the test score. MAE, RMSE and bias below are in centimetres. Positive bias means overprediction.

Test	Seed	Best / ran	MAE	RMSE	R^2	Bias
MH_01	42	11 / 26	3.91	5.60	0.056	+0.38
MH_01	43	11 / 26	3.74	5.58	0.060	+0.88
MH_01	44	18 / 33	3.72	5.20	0.185	+0.35
MH_02	42	24 / 39	5.08	7.81	0.012	-0.03
MH_02	43	40 / 55	4.83	7.58	0.069	+0.12
MH_02	44	17 / 32	5.25	7.61	0.060	-0.65

Across three seeds	MAE (cm)	RMSE (cm)	R^2
MH_01	3.79 ± 0.11	5.46 ± 0.22	0.100 ± 0.073
MH_02	5.05 ± 0.21	7.67 ± 0.12	0.047 ± 0.031

Equal-fold MAE per seed: 42 = 4.49 cm; 43 = 4.29 cm; 44 = 4.49 cm. Their mean and sample standard deviation are 4.42 ± 0.12 cm. Do not pool all windows to calculate the reported R^2. [S2]

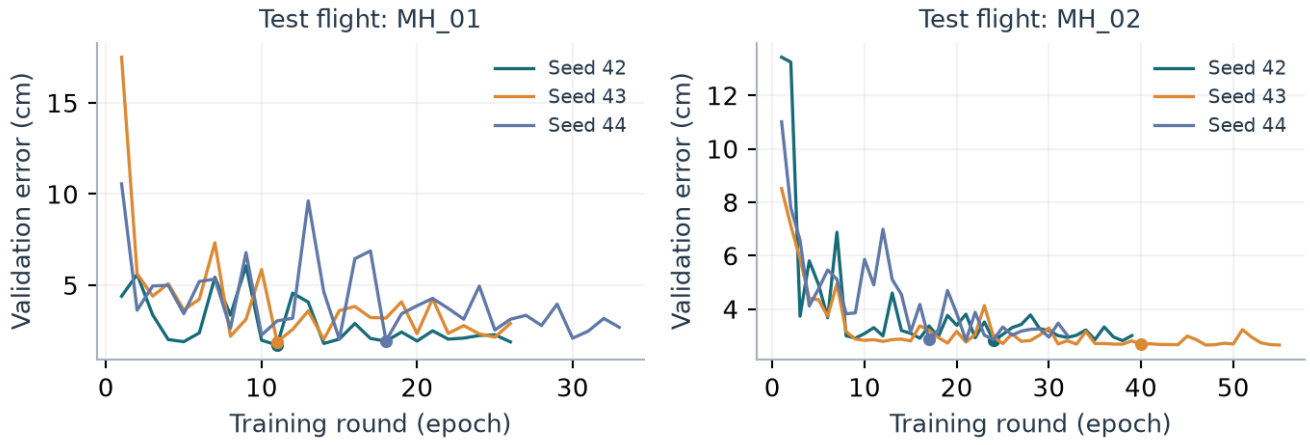


Figure 3. Checking error during training; dots mark the rounds retained by the validation-MSE rule. Different seeds start from different initial weights. All six runs stop before 100 rounds.

What these results support

E5 performs close to the stored mean baseline (4.53 cm) and worse than Lasso (2.50 cm). Mean R^2 is about 0.074, indicating limited predictive improvement over each test flight's own mean. The apparent roughly 0.1 cm MAE difference from the mean baseline is small and does not establish a reliable advantage.

Checking MAE at the retained rounds is 1.72-1.95 cm when testing MH_01 and 2.67-2.85 cm when testing MH_02. Test errors are higher, indicating limited transfer to the other flight. Training loss includes dropout, so it should not be directly compared with checking loss as evidence of overfitting.

5. Insights, confidence and test coverage

Established by the current work

• The camera input pipeline is complete for both flights. All images open and every existing window has 40 frames. Alignment, row order and cached-file checks are implemented.

• The Tier 1 baseline reproduces the old Lasso result, supporting comparability of that experiment. Adding these camera measurements did not improve it.

• The camera-only model learns enough to produce finite predictions, but its scores are weak. More complex inputs alone did not provide better performance.

Possible explanations that have not been proven

• Only two flights are available, and many windows overlap. The image model may learn scene appearance that does not transfer well to the other flight.

• Resizing to 64 x 64 loses detail. Image movement also depends on scene depth, rotation, lighting and texture; it is not a direct measurement of metric velocity or sensor bias.

• A poor standalone image result does not determine whether the image representation supplies complementary information to the IMU model. E6/A3 must test that.

Verification	Recorded outcome
R3 tests	16 passed: 12 camera-feature cases and 4 model/integration cases
Full repository suite	85 passed in the implementation session after the approved fixture adjustment
Notebook	All code cells executed; no error outputs
Fresh report audit	All 10 prediction CSVs (6,560 rows) independently rescored; metadata, labels and saved metrics agree
Integrity checks	Camera source/output checksums and run input checksums match

R3 tests cover exact time boundaries, unchanged row order, missing/gapped data, corrupt images, label-independent image features, movement direction, cache reuse, output sizes, learning on a toy example, repeatability, checkpoint restoration and real R1 integration. Test history is from the completed implementation session; model training and the full suite were not repeated solely to create this report.

> **Limits on claims: seed spread measures training variability, not uncertainty over new flights. E5 trains magnitude only. Vector correction, E6/E7 fusion and A3/A5 evaluation are not completed by this R3 report.**

The Phase 1 assumption remains: labels and physics priors begin windows from ground-truth state. Camera preprocessing does not use those answers, but this study is not a demonstration of fully independent online drone navigation.

6. Instructions for the validation member

Use the accompanying ZIP for score verification. It contains this report, its editable Markdown version, the ten prediction CSVs, metric/CV tables, window labels and timestamps, split manifests, run settings, supporting figures and checksum records. Raw images, NumPy image arrays and model checkpoints are not included; obtain the prepared arrays from the repository to retrain.

Prediction format and naming

Every prediction CSV has seq_id, window_id, y_true, y_pred. Values are in metres. The suffix leave_MH_01_easy means MH_01 is the test flight (720 rows); leave_MH_02_easy means MH_02 is the test flight (592 rows). E5 means seed 42; E5_seed43 and E5_seed44 are the other seeds of the same experiment. Do not count them as three different architectures.

Recommended validation checks

1. Join by seq_id and window_id, confirm complete test-flight coverage and compare y_true against err_mag_m. E5 labels use float32, so allow small rounding differences (1e-6 m is sufficient).

2. Score each CSV with evaluation/metrics.py: MAE = average absolute prediction error; RMSE gives larger misses more weight; R^2 compares squared errors against the test-flight mean; bias = average prediction minus truth.

3. For E5, average the two fold scores within each seed, then report mean and sample standard deviation over the three seeds (ddof=1). Also retain per-fold seed summaries. Do not pool windows or select the best seed.

4. Reproduce the Tier 1 Lasso baseline and preserve the training-selected alpha values. Use camera_tier1_cv.csv to audit selection; do not retune using the test flight.

5. Add the R3 rows to the shared comparison table. Run the first-20%-versus-rest analysis using window timestamps, and evaluate IMU-camera fusion with R4. Vector correction needs separately trained vector outputs.

Evidence map (paths inside the ZIP match the repository)

Source	Files / purpose
[S1] Tier 1	results/diagnostics/camera_tier1_check.csv camera_tier1_cv.csv in the same folder
[S2] E5	results/metrics/camera_e5.csv camera_e5_summary.csv and six *_loss.csv files
[S3] Phase 1	results/metrics/err_mag_m.csv
[S4-S6] Data and setup	results/diagnostics/camera_data_check.csv camera_run_both.json in the same folder data/processed/splits.json and splits_dl.json
Predictions / labels	results/predictions/T1_*.csv and E5*.csv data/processed/windows_<seq>.csv
Audit / instructions	evidence_verification.json; bundle_manifest.json docs/camera_data.md

Appendix. Reproduction and integration

Commands from the repository root

python scripts/verify_camera_data.py

python scripts/make_camera_features.py

python -m pytest tests/test_camera_features.py tests/test_camera_models.py -q

python scripts/train_camera.py

python -m pytest -q

Verification needs raw cam0 images. Preprocessing can reuse the checked interim cache. Training needs the repository's R1 data artifacts plus the prepared camera arrays. Activate the project environment first. Use --tier 1 or --tier 2 for individual experiments; the default runs both with seeds 42/43/44.

To keep a short trial separate from reported outputs:
python scripts/train_camera.py --tier 2 --seeds 42 --epochs 2 \
--output-dir /tmp/drift-sense-camera-smoke

Recorded experiment environment

Component	Version	Component	Version
python	3.11.15	sklearn	1.9.1
numpy	2.4.6	opencv	4.14.0
pandas	3.1.1	torch	2.15.0

Source map

Responsibility	Repository file
Image timing / resizing / summaries	src/vio_drift/features/camera.py
Camera model / training	src/vio_drift/models/dl/camera_cnn.py src/vio_drift/models/dl/camera_train.py
Commands	scripts/verify_camera_data.py scripts/make_camera_features.py scripts/train_camera.py
Review and reporting	notebooks/09_camera_branch.ipynb reports/endsem/r3_camera.tex literature/r3_visual_motion.md

R1/R2 production files and prior result files were preserved. Shared edits were limited to .gitignore, the camera configuration block, data/README.md, and the approved missing-camera test fixture in tests/test_dl_normalize.py. The report entry point remains unchanged for R1 to integrate.

R4 can consume CameraCNNEncoder for a 64-number image representation and feature_names() for the ten image measurements. R5 should retain the negative findings, distinguish implementation correctness from prediction quality, and report the later fusion results separately.